

SIMATS ENGINEERING
DESIGN AND ANALYSIS OF ALGORITHMS
ASSIGNMENT 1 – CO1

**Design, Analyse and Validate an Efficient Search Strategy
for a Large-Scale Student Record System**

Name: Muthupandi C

Registration Number: 192565037

GitHub Repository: <https://github.com/muthupandic84-creator/DESIGN-AND-ANALYSIS-OF-ALGORITHM>

Course: Design and Analysis of Algorithms

Total Marks: 50

SIMATS ENGINEERING
DESIGN AND ANALYSIS OF ALGORITHMS
ASSIGNMENT 1 – CO1

Design, Analyse and Validate an Efficient Search Strategy
for a Large-Scale Student Record System

Course: Design and Analysis of Algorithms

Total Marks: 50

Submission Note

This report is a complete theoretical and implementation-ready draft. Experimental values and screenshots are intentionally not fabricated. Run the supplied benchmark with your own registration number and name, then insert the generated CSV values, graphs and screenshots.

Task 1 – Design the Algorithm

Three meaningfully different strategies are selected: Linear Search, recursive Binary Search, and Hash Table Search.

1. Linear Search (iterative). The records are inspected from the first record to the last until the required registration number is found or the list ends. It requires no preprocessing and is useful as a baseline.

Pseudocode:

LINEAR-SEARCH(A, key)

1. for $i = 0$ to $n-1$
2. compare $A[i]$ with key
3. if $A[i] = \text{key}$, return i
4. return NOT FOUND

2. Binary Search (recursive). The records are sorted first. The middle record is compared with the key. Depending on the comparison, only the left or right half is searched recursively. Preprocessing: sorting, $O(n \log n)$ for a comparison sort.

Pseudocode:

BINARY-SEARCH(A, low, high, key)

1. if $\text{low} > \text{high}$, return NOT FOUND
2. $\text{mid} = \text{floor}((\text{low} + \text{high})/2)$
3. compare $A[\text{mid}]$ with key
4. if equal, return mid
5. if $\text{key} < A[\text{mid}]$, search low ... mid-1 recursively
6. otherwise search mid+1 ... high recursively

3. Hash Table Search. A hash table is constructed from the registration numbers. The key is then located using hash-table membership/lookup. Preprocessing: building the table, expected $O(n)$. Average lookup is expected $O(1)$, although worst-case lookup can degrade to $O(n)$ under severe collisions.

The three strategies are useful because they represent sequential scanning, divide-and-conquer recursion, and direct key-based indexing.

Task 2 – Identify the Basic Operation

Linear Search: the basic operation is comparison of the current record with the target key. It occurs once per inspected record.

Binary Search: the basic operation is comparison between the middle element and the target. Each comparison removes approximately half of the remaining search interval.

Hash Table Search: the basic operation is the key membership/lookup operation. It uses the hash function and table structure to locate the requested key. The expected number of key operations remains approximately constant as n grows.

Task 3 – Mathematical Time-Complexity Function

Let n be the number of student records.

Linear Search:

Best case: 1 comparison.

Worst case: n comparisons.

For an unsuccessful search, n comparisons are required. Therefore $T_L(n) = c \cdot n + d$ in the worst case, giving $\Theta(n)$.

Binary Search:

After sorting, the search interval is approximately halved after every comparison:

$n, n/2, n/4, \dots, 1$.

The number of comparisons k satisfies $n/2^k \leq 1$, so $2^k \geq n$ and $k \geq \log_2 n$. Thus the search cost is $\Theta(\log n)$.

If sorting is performed for an unsorted dataset, the total one-time preparation plus one search is $\Theta(n \log n) + \Theta(\log n) = \Theta(n \log n)$. If the sorted structure is reused for many searches, the preprocessing cost is amortized across the searches.

Hash Table:

Building the table inserts n records, so expected preprocessing is $\Theta(n)$. Expected lookup is $\Theta(1)$, giving $\Theta(n + q)$ for n preprocessing records and q independent searches. Worst-case lookup can be $\Theta(n)$ under pathological collisions.

Task 4 – Best, Average and Worst Case

Linear Search:

Best: target is the first record $\rightarrow$ 1 comparison.

Average: target is typically around the middle for a successful uniformly distributed search $\rightarrow$ approximately $(n+1)/2$ comparisons.

Worst: target is the last record or absent $\rightarrow n$ comparisons.

Binary Search:

Best: target is the middle element $\rightarrow$ 1 comparison.

Average: logarithmic number of comparisons.

Worst: target is absent or reached at the deepest level $\rightarrow \lfloor \log_2 n \rfloor + 1$ comparisons in the usual comparison model.

Hash Table:

Best/expected: target is found directly in its bucket, approximately $O(1)$.

Average/expected: $O(1)$ lookup under a good hash function and controlled load factor.

Worst: collisions can cause $O(n)$ lookup.

Task 5 – Asymptotic Analysis

Linear Search: $O(n)$, $\Omega(1)$, $\Theta(n)$ for worst/typical growth.

Binary Search: $O(\log n)$, $\Omega(1)$, $\Theta(\log n)$ for the search phase.

Hash Table: expected $O(1)$ and expected $\Theta(1)$ lookup; worst-case $O(n)$.

For growth from 1,000 to 1,000,000 records, linear search grows in direct proportion to n , while binary search grows only logarithmically. Hash lookup is expected to remain nearly constant per request after preprocessing. This distinction becomes especially important when the database reaches one million or more records.

Task 6 – Formulate the Recurrence Relation

For recursive binary search, each call works on approximately half the current input.

Input size: n .

Number of recursive calls: one.

Subproblem size: $n/2$.

Work outside the recursive call: constant c for calculating the midpoint and performing the comparison.

Base case: $n \leq 0$ (or $\text{low} > \text{high}$), with constant cost c_0 .

Therefore:

$$T(n) = T(n/2) + c, \text{ for } n > 1$$

$$T(1) = c_0$$

Task 7 – Solve the Recurrence Relation

Using the Master Theorem, $T(n) = aT(n/b) + f(n)$, where $a = 1$, $b = 2$ and $f(n) = \Theta(1)$.

$$n^{\log_b a} = n^{\log_2 1} = n^0 = 1.$$

Therefore $f(n) = \Theta(n^{\log_b a})$, which is Master Theorem Case 2.

Hence:

$$T(n) = \Theta(\log n).$$

Using iteration gives the same result:

$$T(n) = T(n/2) + c$$

$$= T(n/4) + 2c$$

$$= \dots$$

$$= T(n/2^k) + kc.$$

At the base case, $n/2^k = 1$, so $k = \log_2 n$. Thus $T(n) = T(1) + c \log_2 n = \Theta(\log n)$.

Task 8 – Implement and Test the Algorithms

The implementation contains three programs: iterative Linear Search, recursive Binary Search, and Hash Table Search. The benchmark uses the same generated registration-number population for the three comparisons.

Test sizes:

1,000; 10,000; 100,000; 1,000,000.

Required cases:

1. Own registration number: found.
2. Own registration number + 1: not found.

The program prints the student's registration number, name, system date/time, dataset size, result, position and comparison/key-operation count. Run the benchmark before inserting experimental values into this report.

Task 9 – Experimental Performance

The benchmark records input size, algorithm, found/not-found case, comparison/key-operation count and execution time. The output is written to results/results.csv and plot_results.py creates runtime_graph.png and comparisons_graph.png.

Experimental results must be populated from the student's actual run. Do not use invented values.

Required table format:

Algorithm | n | Case | Comparisons/Operations | Time (seconds)

Linear Search | 1,000 | Found | [RUN] | [RUN]

Binary Search | 1,000 | Found | [RUN] | [RUN]

Hash Table | 1,000 | Found | [RUN] | [RUN]

...

Repeat for 10,000, 100,000 and 1,000,000, and for the required not-found test.

Figures:

Figure 9.1: Runtime comparison across dataset sizes.

Figure 9.2: Comparison/key-operation comparison across dataset sizes.

Task 10 – Compare Theory with Experimental Behaviour

The expected experimental trend is that Linear Search grows approximately linearly with n, Binary Search grows logarithmically after sorting, and Hash Table lookup remains approximately constant per query after preprocessing.

Actual timing can differ from theoretical operation counts because of CPU speed, interpreter overhead, memory access, caching, dictionary implementation, sorting cost, and operating-system scheduling. Therefore, the report should compare the shapes/trends rather than expecting exact numerical equality.

After running the benchmark, replace the placeholders with observations from the generated CSV and graphs.

Task 11 – Recommend and Defend the Final Design

For the original student-record problem, Binary Search is a strong choice when the database is maintained in sorted order and predictable logarithmic search behaviour is desired. Hash Table Search can provide faster expected individual lookup after preprocessing, but it requires additional memory and depends on effective hashing.

The final recommendation should be based on the student's measured results. A suitable evidence-based conclusion is:

- Linear Search is simplest and requires no preprocessing, but it does not scale well.

- Binary Search scales well for search operations but requires sorted data.
- Hash Table Search provides expected $O(1)$ lookup after $\Theta(n)$ construction and is particularly attractive when exact-key searches dominate.

The recommendation must cite the student's actual Task 9 measurements rather than simply selecting the lowest Big-O notation.

Task 12 – Day-4 Design Challenge

A. Impact Analysis

The workload changes to 50,000 searches per hour on 1,000,000 records. The database is updated only once per day. Search frequency therefore becomes much more important than before, while the low update frequency makes one-time preprocessing easier to justify.

B. Evaluation of the Original Design

If the original design is Binary Search, its search cost remains $\Theta(\log n)$, but the dataset must be kept sorted. Reusing the sorted structure over 50,000 searches is reasonable because the sorting cost is paid infrequently. However, exact-key hash lookup can reduce expected per-request search work to $O(1)$.

C. Adaptation / Redesign

A practical redesign is to use a hash table as the primary exact-registration lookup structure, rebuilding or updating it when the daily database update occurs. Construction is expected $\Theta(n)$, while each exact-key query is expected $\Theta(1)$. The alternative is to retain a sorted array plus binary search if ordered traversal, range queries, or lower memory overhead are more important.

D. Evidence-Based Defence

The revised decision should consider 1,000,000 records, 50,000 searches/hour, daily updates, preprocessing cost and measured benchmark results. A 50,000-query benchmark should be run against the 1M-record dataset. Report the measured total time and average time/query for the selected design and compare it with the alternatives.

Required Task 12 benchmark:

50,000 searches on 1,000,000 records.

Required Screenshot Captions

Figure 8.1: Linear, Binary and Hash Table search correctness output for $n = 1,000$.

Figure 8.2: Linear, Binary and Hash Table search correctness output for $n = 10,000$.

Figure 8.3: Linear, Binary and Hash Table search correctness output for $n = 100,000$.

Figure 8.4: Linear, Binary and Hash Table search correctness output for $n = 1,000,000$.

Figure 9.1: Runtime comparison generated from the student's results.csv.

Figure 9.2: Comparison/key-operation comparison generated from the student's results.csv.

Figure 12.1: 50,000-search benchmark on 1,000,000 records.

Student Checklist

- Replace YOUR_REG_NO and YOUR_NAME in every source file.
- Generate datasets using the personal seed derived from the last four registration-number digits.

- Run `benchmark.py` and verify found/not-found cases.
- Run `plot_results.py` and include the generated graphs.
- Capture full terminal/IDE screenshots showing name, registration number and date/time.
- Copy actual benchmark values into Task 9.
- Run the 50,000-search / 1M-record benchmark for Task 12.
- Create at least five meaningful Git commits over the week.
- Paste the GitHub repository link on the first page.

EXPERIMENTAL SCREENSHOTS AND GRAPHS

Figure 8.1: Correctness/benchmark output for $n = 1,000$

Figure 8.2: Correctness/benchmark output for $n = 10,000$

Figure 8.3: Correctness/benchmark output for $n = 100,000$

Figure 8.4: Correctness/benchmark output for $n = 1,000,000$

Figure 9.1: Runtime comparison generated from the student's own results.csv.

Figure 9.2: Comparison/key-operation comparison generated from the student's own results.csv.

Figure 12.1: Task 12 benchmark – 50,000 searches on 1,000,000 records.